Chapter 9 worked solutions

Scaling Data, Compute, and Expert Capacity

This guide covers Exercises 1–4, 10, and 12. We count the token positions that a training run can process, follow a token through an expert branch, and trace which vectors move between devices. The companion Jupyter notebook works through Exercises 5–9 and 11 and is also available as a PDF.

1. Spend the same budget

The original run has 12 billion parameters and processes 300 billion token presentations. A **token presentation** is one token position processed during training. Let N count the parameters and let D count those presentations. The chapter estimates the training cost of a dense model as $C \approx 6ND$ **floating-point operations** (FLOPs).

The factor of six comes from counting two operations per weight in a linear layer's forward calculation, a multiplication and an addition. Backpropagation computes gradients with respect to both the layer's inputs and its weights. Each calculation takes about as much matrix work as the forward calculation. Across the dominant dense projections, this gives about six operations per weight per token presentation.

A billion is 10^9. Substitute the actual counts, keeping both factors of 10^9.

$$C \approx 6(12 \times 10^9)(300 \times 10^9) = 21,600 \times 10^{18} = 2.16 \times 10^{22} \text{ FLOPs.}$$

A 24-billion-parameter model uses about twice as many operations per token presentation under this approximation. At the same budget, divide the available operations by the new cost per token.

$$D \approx \frac{C}{6N} = \frac{2.16 \times 10^{22}}{6(24 \times 10^9)} = 1.5 \times 10^{11}.$$

That is **150 billion token presentations**. You can also see this by holding the product ND fixed. Doubling N from 12 to 24 billion requires halving D from 300 to 150 billion.

If we double both original counts, the new model has 24 billion parameters and processes 600 billion tokens. Each token costs about twice as much, and we pay that cost for twice as many tokens. The total is therefore four times the original estimate, or 8.64×10^{22} **FLOPs**.

Run	Parameters in billions	Token presentations in billions	Estimated FLOPs
Original	12	300	2.16×10^{22}
Larger model at the original budget	24	150	2.16×10^{22}
Larger model with twice the original tokens	24	600	8.64×10^{22}

The formula estimates arithmetic for dense training. Sequence length can change attention work without changing the parameter count, and operations such as optimizer updates have separate costs. Converting FLOPs to elapsed time also requires knowing how efficiently the hardware runs the computation.

2. Count what the model sees

Think of a position in a document that contains the word *bridge*. If the training run reads that document three times, the position contributes three presentations. Different positions can also contain the same word. The count D tracks the positions processed on every pass, regardless of whether their words have appeared before.

Run	Positions in one pass	Complete passes	Token presentations D
Corpus A	120,000,000	3	$3 \times 120,000,000 = 360,000,000$
Corpus B	360,000,000	1	$1 \times 360,000,000 = 360,000,000$

With the same dense model, N is also the same, so the estimate for either run is

$$C \approx 6N(360 \times 10^6) = 2.16 \times 10^9 N \text{ FLOPs},$$

where N is the actual parameter count. The two runs have the same estimated cost from $6ND$.

To compare what the models learn, score both on the same **held-out** text, meaning text reserved from training. For each position, measure the negative log of the probability the model assigns to the actual next token, and average over the scored positions. The same number of presentations can supply different training evidence. Corpus A exposes the model to its contexts three times. Corpus B covers more corpus positions and can expose the model to contexts that A never presents. That extra coverage could help if those contexts are relevant to the held-out text. Repeating A could also help the model learn its patterns if A is closer to the text we use for evaluation. The count says how many positions each model processes. The contents and order of those positions, along with how training responds to them, affect the measured loss.

3. Trace a token

The token entering the branch is $\mathbf{x} = (1, 0, 0, 0)$. The router assigns probabilities $(0.1, 0.6, 0.2, 0.1)$ to experts 1 through 4 in that order. An **expert** is a feed-forward network. Its **gate** is the scalar that multiplies all coordinates of its output before those coordinates contribute to the branch update. The exercise labels experts starting at 1. Expert 2 would be index 1 in a Python array.

In Switch top-one routing, expert 2 wins with probability 0.6. For this accepted token, Switch uses the winner's original probability as its gate. The expert branch supplies

$$\mathbf{y} = 0.6(2, -1, 0, 3) = (1.2, -0.6, 0, 1.8).$$

The **residual output** adds the original token vector to the branch update. Here only the first coordinate changes in this addition, because the other three coordinates of $\mathbf{x}$ are zero.

$$\mathbf{x} + \mathbf{y} = (1, 0, 0, 0) + (1.2, -0.6, 0, 1.8) = (2.2, -0.6, 0, 1.8).$$

With the chapter's top-two rule, experts 2 and 3 are selected. Their original probabilities sum to $0.6 + 0.2 = 0.8$. We want the two retained gates to sum to one while keeping their original ratio. Dividing each retained probability by that sum gives $0.6/0.8 = 0.75$ and $0.2/0.8 = 0.25$. The other two experts do not contribute, so their probabilities are outside the denominator.

This is equivalent to applying softmax over the two selected logits. Each original probability is an exponential divided by the sum of all four exponentials. When we divide a selected probability by the sum of the selected probabilities, that common four-expert denominator cancels. The new denominator is the sum of the two selected exponentials.

Expert	Gate	Expert output	Gated contribution
2	0.75	$(2, -1, 0, 3)$	$(1.5, -0.75, 0, 2.25)$
3	0.25	$(0, 1, 4, -1)$	$(0, 0.25, 1, -0.25)$

Adding the two contributions gives the **update** $(1.5, -0.5, 1, 2)$. Adding **x** once gives the **residual output** $(2.5, -0.5, 1, 2)$. The two gate rules are distinct. The two-expert rule redistributes all the weight over the selected pair, while Switch leaves its one gate at 0.6. Applying the same renormalization to just that one gate would give $0.6/0.6 = 1$ and change the Switch calculation.

4. Count a wider bank

One token has six features. Each expert uses an up projection that expands the token from six to 12 features and a down projection that returns it to six. In the chapter's row-vector convention, a projection from a input features to b output features has a weight matrix of shape $[a, b]$. Count the entries of each matrix. Biases are absent, so there are no additional bias entries to count.

Part	Weight shape	Weight count
One expert's up projection	$[6, 12]$	72
One expert's down projection	$[12, 6]$	72
One complete expert	Both matrices	144
Four-expert bank	Four copies of the expert shapes	$4 \times 144 = 576$
Router	$[6, 4]$	24
Whole routed branch	Expert bank and router	**600**

The router produces four scores from the six features. It uses all 24 of its weights to score the four possible choices, even though the token proceeds to only one or two experts.

Under **top-one** routing, one accepted token uses the router and one expert. That is $24 + 144 = 168$ weights. Under **top-two** routing with both choices accepted, the router still scores the same four experts once. The two selected experts each apply their own matrices, so the token uses $24 + 2(144) = 312$ weights. The other experts' matrices remain stored. Both policies therefore have the same 600 stored weights for the branch.

These counts describe a single token's weight use. A batch can send different tokens to different experts and use the entire bank. To estimate operations, we would also account for the activation, router softmax, gate scaling, and any computation on padded slots.

10. Track two devices (optional)

All eight input rows begin on device A. It holds experts 1 and 3, while device B holds experts 2 and 4. Each expert accepts at most three rows. Follow the requests in input order to determine which rows the experts process and which cross between devices. The row numbers below start at 1.

Expert	Device	Requested rows	Accepted rows	Overflow rows
1	A	1	1	None
2	B	2, 3, 4, 5, 6	2, 3, 4	5, 6
3	A	7	7	None
4	B	8	8	None

Expert 2 receives five requests. Rows 2, 3, and 4 arrive first and fill its three slots. Rows 5 and 6 overflow and receive a zero expert update, so the residual keeps their original vectors. Capacity belongs to each expert. Other experts can have free slots without admitting either of the two excess requests for expert 2.

Device A processes **two local rows**, 1 and 7. Device B processes **four remote rows**, 2, 3, 4, and 8. For each remote row, device A sends one width-1,024 input vector to device B. Device B sends a width-1,024 output vector back to device A, where the result contributes to that row's residual output. Each vector carries $1,024 \times 2 = 2,048$ bytes.

Transfer	Number of vectors	Bytes per vector	Vector payload
Dispatch from A to B	4	2,048	8,192 bytes
Return from B to A	4	2,048	8,192 bytes
Total across devices	8	2,048	**16,384 bytes**

The total is 16 KiB, where one KiB is 1,024 bytes. Local rows require no transfer, and we count dispatch only for admitted rows, so the overflow rows add no vector payload. The question counts forward vectors alone. Routing metadata, protocol overhead, padding, and backward-pass traffic fall outside this total.

12. Plan a fair comparison (optional)

Imagine two decoders reading the same passage. One applies the same feed-forward network to every position. The other sends each position to one of eight networks, called experts, and can skip the expert update when its chosen expert is full. We want to compare how well these two decoders predict the next token under a common training compute budget. We will also measure how often the routed model runs out of slots and how long both models take to generate the same number of tokens.

Use four causal decoder blocks with model width $d = 256$, four attention heads, and a maximum context of 256 input positions. Keep the attention projections, position representation, attention masks, normalization, residual additions, embedding, and vocabulary projection the same. Change the feed-forward branch in every block. For the dense branch, use two bias-free matrices with a ReLU between them and intermediate width $1,024$. For the routed branch, use eight bias-free two-matrix ReLU experts of width $1,020$ and a bias-free router that gives each valid input position eight scores.

Branch	Stored weights per block	Weights used for one valid input position
Dense	$2 \cdot 256 \cdot 1,024 = 524,288$	$524,288$
Routed	$8(2 \cdot 256 \cdot 1,020) + 256 \cdot 8 = 4,179,968$	$524,288$ if accepted, $2,048$ if it overflows

For one accepted position, the routed branch uses $522,240$ expert weights and $2,048$ router weights. The router still scores the position if the expert is full. The widths make the counts equal when every request is accepted, while the routed model stores many more weights. Report both the total stored weights and the estimated training compute.

Freeze a deduplicated English text corpus with a manifest of document IDs. Assign groups of duplicate or near-duplicate documents to a single split, using a fixed seed and an 80/10/10 split by group. Train one tokenizer with a vocabulary of 8,192 entries on the training text. Both models use it to tokenize all three splits. Divide each document into the same next-token windows for both models, preserve document boundaries, and mark padded labels as ignored. For each of five paired seeds, start the shared components from copies of the same initial tensors and use the same training batch order. Each run has a fresh AdamW optimizer with betas $(0.9, 0.999)$, weight decay 0.01, and a peak learning rate of 3×10^{-4}. Use the same schedule in both runs, with 5% warmup and cosine decay indexed by the fraction of the training budget spent.

Use Switch-style top-one routing. The largest score selects one expert, and that expert's softmax probability scales its update if the request is accepted. With T valid input positions in a routing group, give each expert capacity $\lceil 1.25T/8 \rceil$. Fix the grouping and acceptance order in advance. Overflow positions keep their residual input and still receive next-token supervision. Use the raw balance term from the chapter with a fixed coefficient of 0.01 as an additional training term for the routed model. Compute validation and test token loss from the next-token objective.

Count forward and backward work consistently for the matrix multiplications in both models. As a first approximation, each weight used in a linear map contributes six operations per input row across the forward and backward passes. Let T_ℓ and A_ℓ be valid input positions and accepted requests in layer ℓ for a batch. With linear maps that process only valid dense or router rows and accepted expert rows, estimate feed-forward and router work by

$$
\begin{aligned}
C_{\text{dense branch}} &\approx 6 \sum_{\ell} T_{\ell}(524, 288), \\
C_{\text{routed branch}} &\approx 6 \sum_{\ell} \left(T_{\ell}(2,048) + A_{\ell}(522, 240)\right).
\end{aligned}
$$

Add the shared work from attention projections, query–key and attention–value products, and the vocabulary projection, using the actual batch shapes. If any of these kernels multiply padded rows or slots, charge the rows they actually execute. Use T_{ℓ} and A_{ℓ} for the statistics on valid requests and useful expert work. Record routing, elementwise operations, and optimizer overhead separately from this matrix multiplication estimate. Choose a common approximate budget, for example 10^{14} FLOPs per run, and stop each run after the first completed update that reaches it. Record the realized estimate and any overshoot from the final batch, along with the update count and tokens processed. Overflow and batch rounding can cause the models to see different numbers of tokens at the same budget. The token counts help explain any loss difference.

Record validation loss at fixed fractions of the budget and commit to evaluating the final checkpoint. After training, run that checkpoint in evaluation mode on the reserved test windows. Sum the negative log probability of the reference over all valid label positions and divide by the total number of valid labels. This gives nats per held-out token, with each scored token counting once. For the routed model, compute the overflow fraction as the total number of rejected requests divided by the total number of requests. Sum both counts across batches and layers before dividing. Report the overall fraction and the per-layer fractions, since an overall average can hide a layer with heavy overflow. Record the training and held-out fractions separately. A token can make a request in each routed layer, so the denominator counts requests rather than distinct token positions.

For generation, use the same accelerator, precision, software, batch of 16 prompts of length 128, and 64 generated tokens per prompt. Use the same greedy decoding and KV-cache procedure. Run to the fixed length so the workloads can be compared directly. Include prompt processing in the end-to-end time. Also report **prefill** time, which covers the prompt pass through selection of the first generated token, and **decode** time, which covers the remaining 63 cached steps. Warm up both models, synchronize the accelerator around each timed interval, and compare medians over repeated runs with the model order alternated. Report the routing capacity policy alongside the timings because batch size and routing group size can change how many expert requests actually run.